MODULE I: INTRODUCTION TO C LANGUAGE

INTRODUCTION

• A computer is an electronic-device capable of manipulating numbers and symbols under the control of a program.
• A program is a sequence of instructions written using a computer programming language to perform a specified task.

A PSEUDOCODE SOLUTION TO PROBLEM

Algorithm

• An algorithm is a step by step procedure to solve a given problem in finite number of steps by
→ accepting a set of inputs and
→ producing the desired output for the given problem
• Example: Write an algorithm to add 2 numbers entered by user.
Step 1: Start
Step 2: Declare variables num1, num2 & sum.
Step 3: Read values of num1 and num2.
Step 4: Add num1 & num2 and assign the result to sum.
sum←num1+num2
Step 5: Display sum
Step 6: Stop

Pseudocode

• Pseudocode is a method of describing the algorithm using a combination of
→ natural language (English like words) and
→ programming language
• This is essentially an intermediate-step towards the development of the actual code.
• Although pseudocode is frequently used, there are no set of rules for its exact writing.
• Psuedocode is used in textbooks and scientific publications to describe various algorithms.
• For Example:
Problem 1: Input two numbers and display their sum.

```
1) read num1, num2
2) find sum = num1 + num2
3) display sum
```

Problem 2: Input the marks and display message "passed" or "failed" based on the marks.

```
1) read marks
2) if student's marks is greater than or equal to 35
        print "passed"
   else
        print "failed"
   endif
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

BASIC CONCEPTS OF A C PROGRAM

• The structure of a C program is shown below:

```
preprocessor directives
void main()
{
        declaration section

        statement-1      // Executable section starts
        statement-2
        statement-3
        statement-4     // Executable section ends
}
```

Preprocessor Directives

• The *preprocessor* accepts the source program and prepare the source program for compilation.

• The preprocessor-statements start with symbol #.

• The normal preprocessor used in all programs is include.

• The #include directive instructs the preprocessor to include the specified file-contents in the beginning of the program.

• For ex:

```
#include<stdio.h>
```

main()

• Every C program should have a function called as main().

• This the first function to be executed always.

• The statements enclosed within left and right brace is called body of the function. The main() function is divided into 2 parts:

1) Declaration Section

• The variables that are used within the function main() should be declared in the declaration-section only.

• The variables declared inside a function are called local-variables.

Ex:

```
int p, t, r;
```

2) Executable Section

• This contains the instructions given to the computer to perform a specific task.

• The task may be to

→ display a message
→ read data or
→ add 2 numbers etc

• Comments are portions of the code ignored by the compiler. The comments allow the user to make simple notes in the source-code.

```
// this is an example for single line comment
/* this is an example
   for multiple line comment */
```

Example: Program to display a message on the screen.

```
#include<stdio.h>
void main()
{
        printf("Welcome to C");
}
```

Output:

```
Welcome to C
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

HOW TO LEARN C LANGUAGE?

• English is a universal language used to communicate with others.
• In the same way, C is a language used to communicate with computer. In other words, C is used to instruct computer to perform particular task.
• The task can be
→ simple task like adding 2 numbers or
→ complex task like building a railway reservation system
• Before you play the game, you should learn rules of the game. So that you can play better and win easily. In the same way, to write C programs, you should learn rules of C language.

STEPS TO LEARN C LANGUAGE

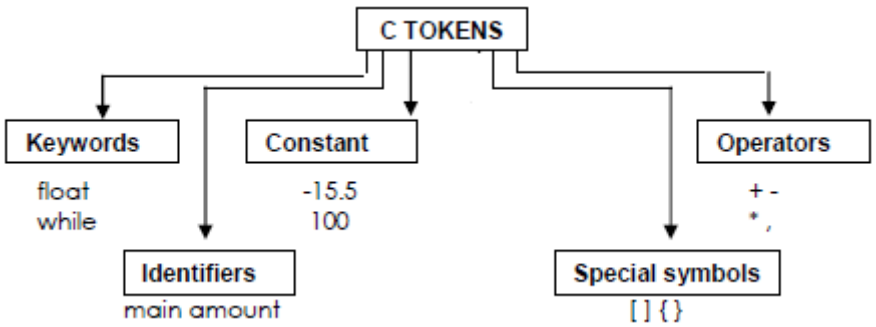
Step 1: Before speaking any language, you should first learn alphabets. In the same way, to learn C language, you should first learn alphabets in C.
Step 2: Then, you should learn how to group alphabets in particular sequence to form a meaningful word. In the same way, in C language, you should learn tokens (i.e. words).
Step 3: Then, you should learn how to group the words in particular sequence to form a meaningful sentence. In the same way, in C language, you should learn instruction (i.e. sentence).
Step 4: Then, you should learn how to group the sentences in particular sequence to form a meaningful paragraph. In the same way, in C language, you should learn program (i.e. paragraph).

CHARACTER SET

• Character-set refers to the set of alphabets, letters and some special characters that are valid in C language.
• For example, the characters in C are:
→ Letters A-X, a-z, both upper and lower
→ Digits 0-9
→ Symbols such as + - * / %
→ White spaces

TOKENS

• A token is a smallest element of a C program.
• One or more characters are grouped in sequence to form meaningful words. These meaningful words are called *tokens.*
• The tokens are broadly classified as follows
→ Keywords ex: if, for, while
→ Identifiers ex: sum, length
→ Constants ex: 10, 10.5, 'a', "sri"
→ Operators ex: + - * /
→ Special symbols ex: [], (), {}



For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

KEYWORDS

• Keywords are tokens which are used for their intended purpose only.

• Each keyword has fixed meaning and that cannot be changed by user. Hence, they are also called reserved-words.

Rules for using keyboards

- Keywords cannot be used as a variable or function.
- All keywords should be written in lower letters.

• Some keywords are as listed below

break	case	char	const	continue	default	do
double	else	float	for	if	int	long
register	return	short	signed	sizeof	struct	switch
typedef	unsigned	void	while			

IDENTIFIER

• As the name indicates, identifier is used to identify various entities of program such as variables, constants, functions etc.

• In other words, an identifier is a word consisting of sequence of

→ Letters
→ Digits or
→ "_"(underscore)

• For ex:

area, length, breadth

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

CONSTANTS

- A constant is an identifier whose value remains fixed throughout the execution of the program.
- The constants cannot be modified in the program.
- For example:

  1, 3.14512 , 'z', "vtunotesbysri"
- Different types of constants are:

1) Integer Constant

- An integer is a whole number without any fraction part.
- There are 3 types of integer constants:
  - i) Decimal constants (0 1 2 3 4 5 6 7 8 9)
    For ex: 0, -9, 22
  - ii) Octal constants (0 1 2 3 4 5 6 7)
    For ex: 021, 077, 033
  - iii) Hexadecimal constants (0 1 2 3 4 5 6 7 8 9 A B C D E F)
    For ex: 0x7f, 0x2a, 0x521

2) Floating Point Constant

- The floating point constant is a real number.
- The floating point constants can be represented using 2 forms:

i) Fractional Form

- A floating point number represented using fractional form has an integer part followed by a dot and a fractional part.
- For ex:

  0.5, -0.99

ii) Scientific Notation (Exponent Form)

- The floating point number represented using scientific notation has three parts namely: mantissa, E and exponent.
- For ex:

  9.86E3 imply $9.86*10^3$

3) Character Constant

- A symbol enclosed within a pair of single quotes(') is called a character constant.
- Each character is associated with a unique value called an ASCII (American Standard Code for Information Interchange) code.
- For ex:

  '9', 'a', '\n'

4) String Constant

- A sequence of characters enclosed within a pair of double quotes(") is called a string constant.
- The string always ends with NULL (denoted by \0) character.
- For ex:

  "9" "a" "sri" "\n"

5) Escape Sequence Characters

- An escape sequence character begins with a backslash and is followed by one character.
- A backslash (\) along with some characters give rise to special print effects by changing (escaping) the meaning of some characters.
- The complete set of escape sequences are:

Escape Sequences	Character
\b	Backspace
\f	Form feed
\n	Newline
\r	Return
\t	Horizontal tab
\v	Vertical tab
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark
\?	Question mark
\0	Null character

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

BASIC DATA TYPES

- The data type defines the type of data stored in a memory-location.
- C supports 3 classes of data types:
  1) Primary data type
     For ex: int, flaot
  2) Derived data types
     For ex: array
  3) User defined data types
     For ex: structure
- C supports 5 primary data types:

1) int

- An int is a keyword which is used to define integers.
- Using int keyword, the programmer can inform the compiler that the data associated with this keyword should be treated as integer.
- C supports 3 different sizes of integer:
  → short int
  → int
  → long int

2) float

- A float is a keyword which is used to define floating point numbers.

3) double

- A double is a keyword used to define long floating point numbers.

4) char

- A char is a keyword which is used to define single character.

5) void

- void is an empty data type. Since no value is associated with this data type, it does not occupy any space in the memory.
- This is normally used in functions to indicate that the function does not return any value.

Range of data types

Data type	Bytes	Range of data type
char	1 bytes	-128 to 127
int	2 bytes	-32, 768 to 32,767
float	4 bytes	3.4E-38 to 3.4E38
double	8 bytes	1.7E-308 to 1.7E308

Qualifiers

- Qualifiers alter the meaning of primary data types to yield a new data type.

Size Qualifiers

- Size qualifiers alter the size of primary data type.
- The keywords long and short are 2 size qualifiers.

For example:

```
long int i; //The size of int is 2 bytes but, when long keyword is
            //used, that variable will be of 4 bytes
short int i; //The size of int is 2 bytes but, when short keyword is
             //used, that variable will be of 1 byte
```

Sign Qualifiers

- Whether a variable can hold positive value, negative value or both values is specified by sign qualifiers.
- Keywords signed and unsigned are used for sign qualifiers.

```
unsigned int a; //unsigned variable can hold zero & positive values only
signed int b; //signed variable can hold zero , positive and negative values
```

Constant Qualifiers

- Constant qualifiers can be declared with keyword const.
- An object declared by const cannot be modified.

```
const int p=20; //the value of p cannot be changed in the program.
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

VARIABLE

• A variable is an identifier whose value can be changed during execution of the program.
In other words, a variable is a name given to a memory-location where the data can be stored.
• Using the variable-name, the data can be
→ stored in a memory-location and
→ accessed or manipulated

Rules for defining a variable

1) The first character in the variable should be a letter or an underscore
2) The first character can be followed by letters or digits or underscore
3) No extra symbols are allowed (other than letters, digits and underscore)
4) Length of a variable can be up to a maximum of 31 characters
5) Keywords should not be used as variable-names

• Valid variables:
a, principle_amount, sum_of_digits

Invalid variables:

```
3fact                              //violates rule 1
sum=  sum-of-digits  62$           //violates rule 3
for int  if                        //violates rule 5
```

Declaration of Variable

• The declaration tells the complier
→ what is the name of the variable used
→ what type of date is held by the variable
• The syntax is shown below:
data_type v_1,v_2,v_3;
where v_1,v_2,v_3 are variable-names
data_type can be int, float or char
• For ex:

```
int a, b, c;
float x, y, z;
```

Initialization of Variable

• The variables are not initialized when they are declared. Hence, variables normally contain garbage values and hence they have to be initialized with valid data.
• Syntax is shown below:

```
data_type var_name=data;
        where data_type can be int, float or char
              var_name is a name of the variable
              = is assignment operator
              data is the value to be stored in variable
```

• For ex:

```
int a=10;
float pi=3.1416;
char c='z';
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

DATA INPUT/OUTPUT FUNCTIONS

- There are many library functions for input and output in C language.
- For ex:
  getch( ), putchar( ), scanf( ), printf( )
- For using these functions in a C-program there should be preprocessor statement #include<stdio.h>.

Input Function

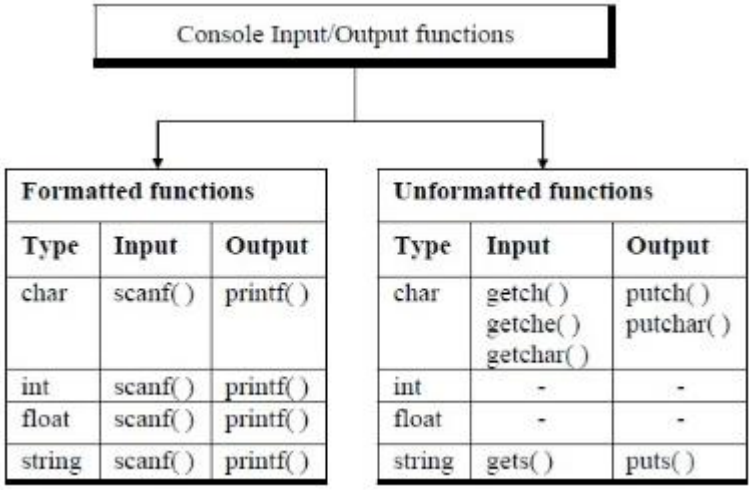
- The input functions are used to read the data from the keyboard and store in memory-location.
- For ex:
  scanf(), getchar(), getch(), getche(), gets()

Output Functions

- The output functions are used to receive the data from memory-locations and display on the monitor.
- For ex:
  printf(), putchar(), putch(), puts()

Types of I/O Functions

- There are 2 types of I/O Functions as shown below:



Formatted functions		
Type	**Input**	**Output**
char	scanf( )	printf( )
int	scanf( )	printf( )
float	scanf( )	printf( )
string	scanf( )	printf( )

Unformatted functions		
Type	**Input**	**Output**
char	getch( ) getche( ) getchar( )	putch( ) putchar( )
int	-	-
float	-	-
string	gets( )	puts( )

UNFORMATTED I/O

getchar() and putchar()

- getchar() is used to
  → read a character from the keyboard and
  → store this character into a memory-location
- You have to press ENTER key after typing a character.
- The syntax is shown below:
  char variable_name = getchar( );
- For ex:
  char z;
  z= getchar( );
- putchar() is used to display a character stored in the memory-location on the screen.

```
#include<stdio.h>
main()
{
        char x;
        char y='n';
        printf("enter one letter terminated by ENTER key \n");
        x = getchar();
        putchar(y);     // same as printf("%c", z);
}
```

Output:

```
enter one letter terminated by ENTER key
m
n
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

getch() and putch()

• getch() is used to read a character from the keyboard without echo(i.e. typed character will not be visible on the screen). The character thus entered will be stored in the memory location.

• putch() is used to display a character stored in memory-location on the screen.

```
#include<stdio.h>
void main()
{
        int  z;
        printf("enter one letter \n ");
        z = getch();
        putch(z);     // same as printf("%c", z)
}
```

Output:

```
enter one letter
m            //m is not visible
m
```

getche()

• getche() is used to read a character from the keyboard with echo(i.e. typed character will be visible on the screen). The character thus entered will be stored in the memory-location.

```
#include<stdio.h>
void main()
{
        int  z;
        printf("enter one letter \n");
        z = getche();
        putch( z);     // same as printf("%c", z)
}
```

Output:

```
enter one letter
m            //m is visible
m
```

gets() and puts()

• gets() is used to

→ read a string from the keyboard and

→ store the string in memory-locations

• puts() is used to display a string stored in memory-locations on the screen.

```
#include<stdio.h>
void main()
{
        char string[20];
        printf("enter a string \n");
        gets(string);
        puts(string);     // same as printf("%s", string);
}
```

Output:

```
enter a string
vtunotesbysri
vtunotesbysri
```

Disadvantage of Unformatted I/O

• It is not possible to read/print any other data except characters i.e. it is not possible to read/print integer numbers, floating point numbers etc.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

FORMATTED I/O

scanf()

- The scanf function does following tasks:
  - → Scan a series of input fields one character at a time
  - → Format each field according to a corresponding format-specifier passed in format-string (format means convert).
  - → Store the formatted input at an address passed as an argument i.e. address-list
- Syntax is shown below:

  n=scanf("format-string", address-list);

  where format-string contains one or more format-specifiers

  address-list is a list of variables. Each variable name must be preceded by &
- For ex:

  n=scanf("%d %f %c", &x, &y, &z);

Format specifiers	**Meaning**
%d	an int argument in decimal
%ld	a long int argument in decimal
%c	a character
%s	a string
%f	a float or double argument
%e	same as %f, but use exponential notation
%o	an int argument in octal (base 8)
%x	an int argument in hexadecimal (base 16)

printf

The printf function does following tasks:

- → Accept a series of arguments
- → Apply to each argument a format-specifier contained in the format-string
- → Output the formatted data to the screen

- The syntax is shown below:

  n=printf("format-string", variable-list);

  where format-string contains one or more format-specifiers

  variable-list contains names of variables
- For ex:

  n=printf("%d %f %c", x, y, z);
- Example: Program to read your age and display the same on the screen.

```
#include<stdio.h>
void main()
{
        int age;
        printf("enter your age: \n");
        scanf("%d", age);
        printf("your age is = %d years ", age);
}
```

Output:

```
enter your age:
21
your age is = 21 years
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

VARIATIONS IN OUTPUT FUNCTION FOR INTEGER AND FLOATS

• Integer and floating-points can be displayed in different formats in C as shown below:

```
#include<stdio.h>
void main()
{
   printf("Case 1:%6d \n",9876);
     //  Prints the number right justified within 6 columns
   printf("Case 2:%3d \n",9876);
     // Prints the number to be right justified to 3 columns but, there are 4
     //  digits so number is not right justified
   printf("Case 3:%.2f \n",987.6543);
     // Prints the number rounded to two decimal places
   printf("Case 4:%.f \n",987.6543);
     // Prints the number rounded to 0 decimal place, i.e., rounded to integer
   printf("Case 5:%e ",987.6543);
     // Prints the number in exponential notation(scientific notation)
}
```

Output:

```
Case 1:    9876
Case 2:9876
Case 3:987.65
Case 4:988
Case 5:9.876543e+002
```

OPERATOR

• An operator can be any symbol like + - * / that specifies what operation need to be performed on the data.

• For ex:

+ indicates add operation

* indicates multiplication operation

Operand

• An operand can be a constant or a variable.

Expression

• An expression is combination of operands and operator that reduces to a single value.

• For ex:

Consider the following expression a+b

here a and b are operands

while + is an operator

CLASSIFICATION OF OPERATORS

Operator Name	**For Example**
Arithmetic operators	+ - * / %
Increment/decrement operators	++ --
Assignment operators	=
Relational operators	< > ==
Logical operators	&& || ~
Conditional operator	?:
Bitwise operators	& | ^
Special operators	[]

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

ARITHMETIC OPERATORS

- These operators are used to perform arithmetic operations such as addition, subtraction,
- There are 5 arithmetic operators:

Operator	Meaning of Operator
+	addition
-	subtraction
*	multiplication
/	division
%	modulos

- Division symbol (/)
  → divides the first operand by second operand and
  → returns the quotient.
  Quotient is the result obtained after division operation.
- Modulus symbol (%)
  → divides the first operand by second operand and
  → returns the remainder.
  Remainder is the result obtained after modulus operation.
- To perform modulus operation, both operands must be integers.
- Program to demonstrate the working of arithmetic operators.

```
#include<stdio.h>
void main()
{
   int a=9,b=4,c;
   c=a+b;
   printf("a+b=%d \n", c);
   c=a-b;
   printf("a-b=%d \n", c);
   c=a*b;
   printf("a*b=%d \n", c);
   c=a/b;
   printf("a/b=%d \n", c);
   c=a%b;
   printf("Remainder when a divided by b=%d ", c);
}
```

Output:

```
a+b=13
a-b=5
a*b=36
a/b=2
Remainder when a divided by b=1
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

INCREMENT OPERATOR

• ++ is an increment operator.
• As the name indicates, increment means increase, i.e. this operator is used to increase the value of a variable by 1.
• For example:
If b=5
then b++ or ++b; // b becomes 6
• The increment operator is classified into 2 categories:
1) Post increment Ex: b++
2) Pre increment Ex: ++b
• As the name indicates, post-increment means first use the value of variable and then increase the value of variable by 1.
• As the name indicates, pre-increment means first increase the value of variable by 1 and then use the updated value of variable.
• For ex:
If x is 10,
then z= x++; sets z to 10
but z = ++x; sets z to 11

Example: Program to illustrate the use of increment operators.

```
#include<stdio.h>
void main()
{
        int x=10,y = 10, z ;
        z= x++;
        printf(" z=%d x= %d\n", z, x);
        z = ++y;
        printf(" z=%d y= %d", z, y);
}
```

Output:

```
z=10 x=11
z=11 y=11
```

DECREMENT OPERATOR

• -- is a decrement operator.
• As the name indicates, decrement means decrease, i.e. this operator is used to decrease the value of a variable by 1.
• For example:
If b=5
then b-- or --b; // b becomes 4
• Similar to increment operator, the decrement operator is classified into 2 categories:
1) Post decrement Ex: b--
2) Pre decrement Ex: --b
• For ex:
If x is 10,
then z= x--; sets z to 10,
but z = --x; sets z to 9.

Example: Program to illustrate the use of decrement operators.

```
void main()
{
        int x=10,y = 10, z ;
        z= x--;
        printf(" z=%d x= %d\n", z, x);
        z = --y;
        printf(" z=%d y= %d", z, y);
}
```

Output:

```
z=10 x=9
z=9 y=9
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

ASSIGNMENT OPERATOR

- The most common assignment operator is =.
- This operator assigns the value in right side to the left side.
- The syntax is shown below:
  variable=expression;
- For ex:
  c=5; //5 is assigned to c
  b=c; //value of c is assigned to b
  5=c; // Error! 5 is a constant.
- The operators such as +=,*= are called shorthand assignment operators.
- For ex,
  a=a+10: can be written as a+=10;
- In the same way, we have:

Operator	Example	Same as
-=	a-=b	a=a-b
*=	a*=b	a=a*b
/=	a/=b	a=a/b
%=	a%=b	a=a%b

RELATIONAL OPERATORS

- Relational operators are used to find the relationship between two operands.
- The output of relational expression is either true(1) or false(0).
- For example
  a>b //If a is greater than b, then a>b returns 1 else a>b returns 0.
- The 2 operands may be constants, variables or expressions.
- There are 6 relational operators:

Operator	Meaning of Operator	Example
>	Greater than	5>3 returns true (1)
<	Less than	5<3 returns false (0)
>=	Greater than or equal to	5>=3 returns true (1)
<=	Less than or equal to	5<=3 return false (0)
==	Equal to	5==3 returns false (0)
!=	Not equal to	5!=3 returns true(1)

- For ex:

Condition	Return values
2>1	1 (or true)
2>3	0 (or false)
3+2<6	1 (or true)

- Example: Program to illustrate the use of all relational operators.

```
#include<stdio.h>
void main()
{
        printf("4>5 : %d  \n", 4>5);
        printf("4>=5 : %d \n", 4>=5);
        printf("4<5 : %d \n", 4<5);
        printf("4<=5 : %d \n", 4<=5);
        printf("4==5 : %d \n", 4==5);
        printf("4!=5 : %d ", 4!=5);
}
```

Output:

```
4>5 : 0
4>=5 : 0
4<5 : 1
4<=5 :1
4==5 : 0
4!=5 : 1
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

LOGICAL OPERATORS

- These operators are used to perform logical operations like negation, conjunction and disjunction.
- The output of logical expression is either true(1) or false(0).
- There are 3 logical operators:

Operator	Meaning	Example
&&	Logical AND	If c=5 and d=2 then ((c==5) && (d>5)) returns false.
||	Logical OR	If c=5 and d=2 then ((c==5) || (d>5)) returns true.
!	Logical NOT	If c=5 then !(c==5) returns false.

- All non zero values(i.e. 1, -1, 2, -2) will be treated as true.
  While zero value(i.e. 0) will be treated as false.

Truth table

A	B	A&&B	A||B	!A
0	0	0	0	1
0	1	0	1	
1	0	0	1	0
1	1	1	1	

- Example: Program to illustrate the use of all logical operators.

```
#include<stdio.h>
void main()
{
        clrscr ( );
        printf("7 && 0 : %d \n", 7 && 0 );
        printf("7 || 0 : %d \n", 7 || 0 );
        printf(" !0 : %d", !0 );
}
```

Output:

```
7 && 0 : 1
7 || 0 : 1
!0 : 1
```

- Example: Program to illustrate the use of both relational & Logical operators.

```
#include<stdio.h>
void main()
{
        printf("5>3 && 5<10 : %d \n", 5>3 && 5<10);
        printf(" 8<5 || 5= =5 : % d  \n", 8<5 || 5==5);
        printf("!(8 = =8) : %d ", !(8==8) ;
}
```

Output:

```
5>3 && 5<10 : 1
8<5 || 5= =5 : 1
!(8 = =8) : 0
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

CONDITIONAL OPERATOR

- The conditional operator is also called ternary operator it takes three operands.
- Conditional operators are used for decision making in C.
- The syntax is shown below:

(exp1)? exp2: exp3;

where exp1 is an expression evaluated to true or false;

If exp1 is evaluated to true, exp2 is executed;

If exp1 is evaluated to false, exp3 is executed.

Example: Program to find largest of 2 numbers using conditional operator.

```
#include<stdio.h>
void main()
{
        int a,b, max ;
        printf(" enter 2 distinct numbers \n");
        scanf("%d %d", &a, &b);
        max=(a>b)? a : b;
        printf(" largest number =%d ", max);
}
```

Output:

```
enter 2 distinct numbers
3 4
largest number = 4
```

BITWISE OPERATORS

- These operators are used to perform logical operation (and, or, not) on individual bits of a binary number.
- There are 6 bitwise operators:

Operators	**Meaning of operators**
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement
<<	Shift left
>>	Shift right

Truth Table

A	B	A&B	A|B	A^B	~A
0	0	0	0	0	1
0	1	0	1	1	
1	0	0	1	1	0
1	1	1	1	0	

- Ex for ~ (bitwise complement)

a = 13 0000 1101

~a= 11110010

Ex for & (bitwise AND)

a = 13 0000 1101

b = 6 0000 0110

a & b 0000 0100

- Ex for || (bitwise OR)

a = 13 0000 1101

b = 6 0000 0110

a | b 0000 1111

Ex for ^ (bitwise xor)

a = 13 0000 1101

b = 6 0000 0110

a ^ b 0000 1011

- The operator that is used to shift the data by a specified number of bit positions towards left or right is called shift operator.
- The syntax is shown below for <<

b=a << num;

where a is value to be shifted

num is number of bits to be shifted

The syntax is shown below for >>

b=a >> num;

- Ex for <<(left shift):

a = 13 0000 1101

b=a<<1 0001 1010

Ex for >>(right shift):

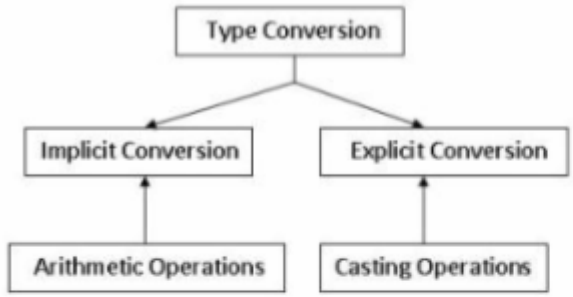
a = 13 0000 1101

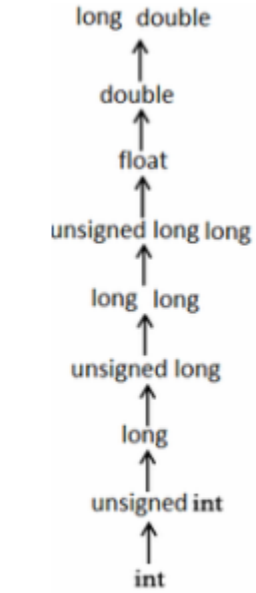
b=a<<1 0000 0110

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

TYPE CONVERSION

- Type conversion is used to convert data of one type to data of another type.
- Type conversion is of 2 types as shown in below figure:





datatype hierarchy

IMPLICIT CONVERSION

- If a compiler converts one type of data into another type of data automatically, it is known as implicit conversions.
- There is no data loss in implicit conversion.
- The conversion always takes place from lower rank to higher rank.
  For ex, int to float as shown in the above datatype hierarchy.
- For ex:
  int a = 22, b=11;
  float c = a; //c becomes 21.000000
  float d=b/c=11/22.000000=11.000000/22.000000=0.500000
- If one operand type is same as other operand type, no conversion takes place and type of result remains same as the operands i.e. int+int=int
  float+float=float
- Conversion rules are as follows:
  → If either operand is long double, convert the other to long double.
  → Otherwise, if either operand is double, convert the other to double.
  → Otherwise, if either operand is float, convert the other to float.
  → Otherwise, convert char and short to int.
  → Then, if either operand is long, convert the other to long.
- Example: Program to illustrate implicit conversion.

```
#include<stdio.h>
void main()
{
        int a = 22, b=11;
        float d ;
        d=b/c;
        printf("d Value is : %f ", d );
}
```

Output:

```
d Value is : 0.500000
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

EXPLICIT CONVERSION

• When the data of one type is converted explicitly to another type with the help of some pre-defined functions, it is called as explicit conversion.
• There may be data loss in this process because the conversion is forceful.
• The syntax is shown below:

```
data_type1 v1;
data_type2 v2= (data_type2) v1;
        where  v1 can be expression or variable
```

• For ex:

```
float b=11.000000;
int c = 22;
float d=b/(float)c=11.000000/22.000000=0.500000
```

• Example: Program to illustrate explicit conversion.

```
#include<stdio.h>
void main()
{
        float b=11.000000;
        int c = 22;
        float d;
        d=b/(float)c;
        printf("d Value is : %f ", d );
}
```

Output:

```
d Value is : 0.500000
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

THE PRECEDENCE OF OPERATORS

• The order in which different operators are used to evaluate an expression is called precedence of operators.

Precedence Table

	Description	Represented By
1	Parenthesis	() []
1	Structure Access	. ->
2	Unary	! ~ ++ -- - * &
3	Mutiply, Divide, Modulus	* / %
4	Add, Subtract	+ -
5	Shift Right, Left	>> <<
6	Greater, Less Than, etc	> < =
7	Equal, Not Equal	== !=
8	Bitwise AND	&
9	Bitwise Exclusive OR	^
10	Bitwise OR	|
11	Logical AND	&&
12	Logical OR	||
13	Conditional Expression	?:
14	Assignment	= += -= etc
15	Comma	,

• Example to understand the operator precedence available in C programming language.

```
#include<stdio.h>
void main()
{
  int a = 20;
  int b = 10;
  int c = 15;
  int d = 5;
  int e;
  e = (a + b) * c / d;            // ( 30 * 15 ) / 5
  printf("Value of (a + b) * c / d is : %d \n",  e );
  e = ((a + b) * c) / d;          // (30 * 15 ) / 5
  printf("Value of ((a + b) * c) / d is  : %d \n" ,  e );
  e = (a + b) * (c / d);          // (30) * (15/5)
  printf("Value of (a + b) * (c / d) is  : %d \n",  e );
  e = a + (b * c) / d;            //  20 + (150/5)
  printf("Value of a + (b * c) / d is  : %d " ,  e );
}
```

Output:

```
Value of (a + b) * c / d is : 90
Value of ((a + b) * c) / d is  : 90
Value of (a + b) * (c / d) is  : 90
Value of a + (b * c) / d is  : 50
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/